

Sistemas Multi-Agente y Function Calling

Hackathon FISC AI AGENTS · UTP, salon 3-303

Dos horas. Sales con un orquestador con subagentes, salida estructurada, paralelo y tolerancia a fallos — y con el esqueleto de tu MVP.

**Un subagente es un agente normal,
llamado desde dentro de una herramienta
de otro agente.**

Eso es todo. No hay framework magico.

Dos modos

MODO=test

No usa red ni llave. Verifica que tu código corre.

MODO=gemini

Respuestas de verdad. Necesita GEMINI_API_KEY.

Armas en test, pruebas en gemini. Esa disciplina es la diferencia entre llegar al sábado con cuota y sin cuota.

Cada quien necesita SU PROPIA llave.

La capa gratis da 5-15 peticiones por minuto. Una llave compartida entre 30 se muere en el primer ejercicio.

Un agente, 15 herramientas, 30 pasos. ¿Que se rompe?

- **El contexto.** El paso 30 carga los 29 anteriores.
- **Las decisiones.** Elegir entre 15 falla mas que entre 3.
- **Los permisos.** Si tiene escribir, la tiene siempre.
- **El costo.** El modelo caro listando carpetas.

Los subagentes son la respuesta a estas cuatro. No a una idea de orden. Y hoy las medimos, no las afirmamos.

1

La descripcion ES el prompt

18 min

El modelo no ve tu código

- El nombre de la función
- Los tipos de los parámetros
- El docstring

*Esa es **toda** la interfaz. Tu docstring no es documentación: es prompt.*

Esto no da error

```
@agente.tool_plain
def buscar_texto(patron: str) -> str:
    """Busca cosas."""
    return _buscar(patron)
```

Da una llamada equivocada hecha con total confianza. Eso es lo peligroso.

Un docstring util dice tres cosas

- Que devuelve, y en que formato
- Que **NO** hace / que no acepta
- Que significa el resultado vacio

La tercera es la que nadie escribe. Si 'Sin coincidencias' significa no esta y no fallo, dilo, o el modelo reintentará en bucle.

Tu turno · ejercicio 1

1. Corre el archivo. Mira que hace con el docstring malo.
2. Arreglalo. Corre otra vez y compara.
3. Escribe una herramienta desde cero. El cuerpo es una linea; el trabajo es el docstring.
4. Pide un archivo que no existe. Fijate: el agente LEE el error y reintenta.

```
python ejercicios/01_herramientas.py
```

2

Que hace `run_sync` por debajo

14 min

run_sync no es magia. Es este bucle.

```
mensajes = [objetivo]
mientras no termine:
    respuesta = modelo(mensajes, herramientas)
    si respuesta no pide herramientas: listo
    por cada herramienta pedida:
        salida = ejecutar(herramienta)
        mensajes.append(salida)    <- - SIN ESTO no avanza
```

Dos consecuencias

- El modelo **no tiene memoria** entre llamadas. En cada vuelta se le reenvia el historial completo. La lista es la memoria.
- El paso 10 carga los 9 anteriores. El costo crece mas rapido que los pasos.

El tope de peticiones es lo unico que separa un bug de una factura.

`UsageLimits(request_limit=6, tool_calls_limit=5)`. Ponlo antes de la primera corrida, no despues del susto.

Tu turno · ejercicio 2

1. Corre y mira el historial completo impreso.
2. Cuenta los ModelResponse. Cada uno es una llamada que pagaste.
3. Baja request_limit a 2 y corre. Lee el error.

```
python ejercicios/02_bajo_el_capo.py
```

**Si tu MVP necesita 30 pasos,
¿que le pasa al historial?
¿Y al costo?**

Esa respuesta es por que existen los subagentes.

3

El bucle multi-agente

30 min · el central

El patron completo

```
@orquestador.tool
async def explorar(ctx, pregunta: str) -> str:
    resultado = await explorador.run(
        pregunta, usage=ctx.usage)
    return resultado.output
```

Para el orquestador es una herramienta cualquiera. No sabe que por dentro corrio otro agente con su propio bucle.

Cuatro razones para separar un subagente

- **Aisla contexto.** Quema 40.000 tokens y devuelve 200 palabras. Esos 40.000 nunca entran al padre. ← la principal
- **Menos herramientas por decision.** Elegir entre 3, no entre 15.
- **Modelo por tarea.** Lo mecanico, en algo mas barato.
- **Permisos.** No puede hacer lo que no tiene.

El aislamiento, en numeros

explorador	quemo	344 tokens	->	devolvio	796 caracteres
escritor	quemo	166 tokens	->	devolvio	48 caracteres

Al historial del orquestador solo entro la segunda columna. En MODO=test la relacion sale al revés: TestModel repite en vez de resumir.

El explorador no escribe porque no tiene la herramienta.

Un prompt que dice 'no borres nada' es una recomendacion. Una funcion que no existe es una garantia.

Tu turno · ejercicio 3

1. Corre tal cual. Ya funciona con UN subagente.
2. **Construye el escritor:** el Agent, su unica herramienta, y registralo con @orquestador.tool.
3. Cambia el OBJETIVO a algo que necesite los dos. Abre sandbox/reporte.md.
4. Empeora el docstring de explorar. ¿Lo sigue usando?
5. Quita usage=ctx.usage y corre. ¿Que le paso al conteo?

```
python ejercicios/03_delegacion.py
```

**El costo real de delegar no es el token.
Es que el padre le encarga al hijo
algo peor de lo que cree.**

Tarea 6: mandale un encargo vago. Nadie falla, ningun 429, y el reporte sale mal. Cuando tu MVP falle el sabado y los dos agentes "funcionen", revisa el encargo primero.

4

La salida estructurada

18 min

El subagente devuelve un objeto, no prosa

```
class Hallazgo(BaseModel):  
    archivo: str  
    linea: int  
    sintoma: str  
    causa: str | None = None  
  
explorador = Agent(modelo(), output_type=Hallazgo, ...)
```

resultado.output ya no es un string. Es un Hallazgo, validado por Pydantic antes de que tu código lo vea.

Que cambia

- **El padre puede comprobar.** `if not hallazgo.causa` es una linea. Con prosa hace falta otro modelo.
- **El modelo ya no elige el formato.** Si no encaja, Pydantic AI lo hace reintentar.
- **Se acaban los parseos.** Ninguna regex sobre espanol a las 2 a.m.
- **Es mas barato.** Cuatro campos pesan menos que 80 palabras.

El esquema ES la instruccion.

Agregas un campo y el modelo lo llena sin que toques el prompt. Antes la interfaz era el docstring; ahora tambien es el tipo.

Un campo obligatorio que el modelo no sabe es una alucinacion garantizada.

Por eso causa es opcional, y las instrucciones dicen "si no la encuentras, dejala en null".
Lo opcional le da permiso de no saber.

Tu turno · ejercicio 4

1. Corre. Mira el tipo que devuelve: Hallazgo, no str.
2. **Escribe la comprobacion:** si no hay causa, devuelve INCOMPLETO y deja que el orquestador decida.
3. Agrega un campo al esquema. No toques el prompt.
4. Compara: `len(str(hallazgo))` contra 400 caracteres de prosa.

```
python ejercicios/04_salida_estructurada.py
```

5

En paralelo, y cuando algo falla

15 min

Si no dependen uno del otro, no los esperes

```
a, b = await asyncio.gather(  
    explorador_codigo.run(pregunta, usage=ctx.usage),  
    explorador_docs.run(pregunta, usage=ctx.usage),  
)
```

Sin await en cada uno: gather los espera. Cuesta lo que el mas lento, no la suma.

El reloj baja. El gasto no.

Son las mismas peticiones. Y peor: dos rafagas simultaneas se acercan al limite de tasa mas rapido que dos espaciadas. Paraleliza de dos en dos, no de diez en diez.

Un hijo sin try/except mata la corrida entera.

Y el sabado un subagente se va a caer: un 429, una ruta mala, un timeout. Devuelve el error como TEXTO, igual que en el ejercicio 1. Si el padre lo LEE, decide.

Reintentar o rendirse bien

- **Temporal** (429, timeout): reintentar, con espera.
- **Permanente** (ruta mala, esquema imposible): ríndete y dilo. Reintentar quema dos peticiones en vez de una.
- ¿Sirve una respuesta a medias? Eso es **producto**, y va en el prompt del orquestador. Si no se lo dices, se lo inventa.

Toda delegación va protegida. No la que crees riesgosa: todas.

Tu turno · ejercicio 5

1. Corre. Mira los dos relojes: en serie y en paralelo.
2. **Completa en_paralelo con asyncio.gather.**
3. Descomenta el raise. Mira morir la corrida completa.
4. Atrapa el fallo y devuelvelo como texto. **En las DOS delegaciones** — el orquestador llama a en_serie primero.

```
python ejercicios/05_paralelo_y_fallos.py
```

6

El esqueleto de TU MVP

12 min

Primero en papel. Sin teclado.

- ¿Que hace tu MVP, en UNA frase?
- Por cada subagente: que hace, que herramientas SI tiene, y **que herramientas NO tiene.**
- ¿Que razon de las cuatro lo justifica?
- ¿Cual es tu limite de peticiones?

La tercera pregunta del segundo punto es la que importa. Si no la puedes contestar, todavia no sabes que vas a construir.

**Si no aisla contexto, no acota permisos,
no reduce decisiones y no cambia de modelo:
no es un subagente. Es una funcion.**

Multi-agente no es gratis. Cada salto son peticiones, latencia, y una traduccion mas donde se pierde informacion.

Tu turno · ejercicio 6

1. Contesta las cuatro preguntas en papel.
2. Llena los TODO con TU MVP, no con el ejemplo de archivos.
3. Pon MIS_LIMITES antes de la primera corrida.

`python ejercicios/06_mi_mvp.py` · esto se lo llevan y lo siguen el sabado

Las dos ideas que se llevan

- **Un subagente es una herramienta**, y su descripción es un prompt. Si el orquestador no lo usa, casi siempre el problema es el docstring.
- **Lo que garantiza algo es el código, no el prompt.** Ningún agente sale del sandbox porque hay una función que rechaza la ruta.

¿Que van a construir?

Una frase cada equipo.